NestJS Fundamentals Project

Explanation

شرح مشروع NestJS

Fundamentals

---

1. الفكرة العامة للمشروع

بسيط جدًا لتعلّم أساسيات بناء NestJS مشروع عبارة عن تطبيق

Users الأساسي منه هو إدارة بيانات المستخدمين REST API.

: Create, Get All, Get endpoints مجموعة

One, Update, Delete.

المشروع لا يستخدم قاعدة بيانات حاليًا. بدلًا من ذلك، يتم حفظ البيانات داخل مصفوفة في

هذا يعني أن UserController. users اسمها

أي بيانات يتم إضافتها ستضيع بمجرد إعادة تشغيل السيرفر

1. Project Overview

This is a very small `NestJS` project built to learn the basics of a `REST` `API`. Its main purpose is handling user data through endpoints such as `Create`, `Get All`, `Get One`, `Update`, and `Delete`.

The project does not use a database yet. Instead, data is stored in an in-

memory array called `users` inside `UserController`. That means all

data is lost whenever the server restarts.

2. الهيكل الأساسي للمشروع

الملف	وظيفته
`src/main.ts`	نقطة تشغيل التطبيق. يبدأ السيرفر ويشغل `AppModule`.
`src/app.module.ts`	الموديول الرئيسي للتطبيق ويقوم بضم `UserModule`.
`src/user/user.module.ts`	الموديول الخاص بجزء المستخدمين، ويسجل

	`UserController`.
`src/user/user.controller.ts`	هنا يوجد المنطق الأساسي والـ routes الخاصة بالمستخدمين.
`src/user/user.entity.ts`	تعريف شكل كائن المستخدم داخل النظام.
`src/user/dtos/create-user.dto.ts`	تعريف البيانات المطلوبة عند إنشاء مستخدم جديد.
`src/user/dtos/update-user.dto.ts`	تعريف البيانات المسموح تعديلها، وكلها اختيارية بسبب `PartialType`.

2. Core Structure

File	Purpose
`src/main.ts`	The application entry point. It boots the server and loads `AppModule`.
`src/app.module.ts`	The root module of the application. It imports `UserModule`.
`src/user/user.module.ts`	The user feature module. It registers `UserController`.
`src/user/user.controller.ts`	This file contains the main user routes and business logic.
`src/user/user.entity.ts`	Defines the shape of a user object in the app.

`src/user/dtos/create-user.dto.ts`	Defines the required data when creating a new user.
`src/user/dtos/update-user.dto.ts`	Defines the updatable fields, all optional because of `PartialType`.

3. كيف يبدأ التطبيق؟

باستخدام	Nest	إنشاء تطبيق	`main.ts`	

PORT لاستماع على البورت	3000	أو قيمة	`NestFactory.create(AppModule)`

البيئة.

routes يعني أن كل	`UserModule`.	استيراد	`app.module.ts`	

جودة في جزء المستخدمين أصبحت جزءًا من التطبيق.

يعرف أن Nest التالي UserController. تسجيل user.module.ts

controller. الـ هب لهذا /users صة بـ requests

3. How the App Starts

In `main.ts`, Nest creates the application with `NestFactory.create(AppModule)`

and starts listening on port `3000` or `process.env.PORT`.

In `app.module.ts`, the root module imports `UserModule`, which makes user routes

available in the app.

In `user.module.ts`, `UserController` is registered, so Nest knows where to send

requests that target `/users`.

4. الوصول الـ Route داخل

UserController

يعني أن هذا الـ controller

```
@Controller('users')
```

/users. ل عن كل المسارات التي تبدأ بـ

أمثلة :

- `@Get()` | | `GET /users` |

- `@Get(':id')` | | `GET /users/:id` |

- `@Post()` | | `POST /users` |

- `@Delete(':id')` | | `DELETE /users/:id` |

4. Route Base in UserController

The line `@Controller('users')` means this controller handles every

route that starts with `/users`.

- `@Get()` becomes `GET /users`

- `@Get(':id')` becomes `GET /users/:id`

- `@Post()` becomes `POST /users`

- `@Delete(':id')` becomes `DELETE /users/:id`

5. شرح الـ Entity والـ DTOs

5.1 UserEntity

	user.entity.ts	يحدد شكل المستخدم داخل النظام

id, username, email, country.

5.2 CreateUserDto

POST	ملف يحدد البيانات المطلوبة لإنشاء مستخدم جديد. عند عمل
إرسال	username, email, country.
/users	

5.3 UpdateUserDto

DTO		م	PartialType(CreateUserDto)،

تصبح اختيارية في	`CreateUserDto`	عناه أن نفس الحقول الموجودة في

التحديث	

5. Entity and DTOs

5.1 UserEntity

`user.entity.ts` defines the internal user shape: `id`, `username`, `email`,

`country`.

5.2 CreateUserDto

This file defines the required data to create a user. A `POST /users`

request should send: `username`, `email`, and `country`.

5.3 UpdateUserDto

This DTO extends `PartialType(CreateUserDto)`, meaning the same

fields from `CreateUserDto` are reused but become optional for

updates.

6. شرح العمليات الأساسية

6.1 Get All Users

Route: `GET /users`

كما هي. إذا لم يتم إنشاء أي `this.users` ع المصفوفة `find()` ثود

تخدمين فسترجع مصفوفة فارغة.

رة: قراءة كل المستخدمين الموجودين حاليًا في الذاكرة.

6.2 Get One User

Route: `GET /users/:id`

id	ث داخل المصفوفة عن مستخدم يحمل نفس الـ	`findOne(id)`	ثود

`find()`.	نخدام

`undefined`.	جد المستخدم فسيتم إرجاعه. وإذا لم يوجد فستكون النتيجة

6.3 Create User

Route: `POST /users`

ثم ، CreateUserDto النوع body قبل create() ثود

ئ كائن مستخدم جديد وتضيف له id طة uuid().

لحفظ المستخدم في المصفوفة، this.users.push(newUser) ذلك يتم تنفيذ

.تم إرجاع المستخدم الجديد

6.4 Delete User

Route: DELETE /users/:id

للاحتفاظ بكل المستخدمين ما عدا filter() خدم remove(id) ثود

id. ستخدم الذي يحمل نفس الـ

لذلك الاستجابة تكون بحالة @HttpCode(HttpStatus.NO_CONTENT) إضافة

بدون	204	body.

6. Main Operations

6.1 Get All Users

Route: `GET /users`

The `find()` method returns the `this.users` array as-is. If no users have been created, it returns an empty array.

Idea: read all currently stored users from memory.

6.2 Get One User

Route: `GET /users/:id`

The `findOne(id)` method searches the array using `find()` for a matching

user `id`.

If a user is found, it is returned. Otherwise, the method returns

`undefined`.

6.3 Create User

Route: `POST /users`

The `create()` method receives a body of type `CreateUserDto`, then builds

a new user object and adds an `id` using `uuid()`.

After that, it stores the user with `this.users.push(newUser)` and returns

the newly created user.

6.4 Delete User

Route: `DELETE /users/:id`

The `remove(id)` method uses `filter()` to keep all users except the one

whose `id` matches the route parameter.

Because of `@HttpCode(HttpStatus.NO_CONTENT)`, the response status is

204 with no response body.

7. الداخلي للمشروع Logic

- واحد فقط لتنفيذ Controller المشروع يعتمد على

المنطق

- Service layer لا يوجد هنا ذلك كل Business logic

داخل UserController.

- يوجد Database أو Repository.

- توليد id لـ uuid. استخدام مكتبة

- private users: البيانات الوحيد هو المصفوفة

```
UserEntity[] = [].
```

أصبح controller ، البنية مناسبة للتعلم والبداية، لكنها ليست الأفضل لمشروع حقيقي لأن الـ

ؤولًا عن أكثر من مهمة: استقبال الطلبات، وتنفيذ المنطق، وإدارة البيانات.

7. Internal Logic

•The project currently depends on a single controller for most

behavior.

- There is no service layer yet, so business logic lives inside

---

`UserController`.

•There is no database and no repository layer.

- User IDs are generated with the `uuid` package.

- The only data source is `private users: UserEntity[] = []`.

> This structure is fine for learning, but it is not ideal for a real project because the

> controller is handling request routing, logic, and data management together.

8. ملاحظات مهمة على الكود الحالي

- لا يوجد Validation للـ body مثل التأكد من صحة البريد الإلكتروني أو وجود الحقول

- فالنتيجة `GET /users/:id` طلب مستخدم غير موجود في

بدل . 404 undefined

- غير موجود قد يحدث سلوك إذا id الـ PATCH

findIndex().ن لأن الكود لا يتحقق من

- لا يوجد تأكيد إن كان المستخدم موجودًا قبل DELETE

الحذف

- البيانات تضيع بعد إعادة تشغيل التطبيق لأنها محفوظة في الذاكرة فقط

8. Important Notes About the Current Code

- There is no request validation yet, such as checking email format or required fields.

- If a user is missing in `GET /users/:id`, the method returns

`undefined` instead of a `404`.

- In `PATCH`, if the ID does not exist, the current `findIndex()` flow is

unsafe because it is not checked.

- In `DELETE`, there is no confirmation that the user existed before

deletion.

•All data disappears after a server restart because it is stored only

in memory.

9. أمثلة سريعة للطلبات

Create

```
POST /users
```

```
{ "username": "Ali", "email": "ali@mail.com", "country":
```

```
"Egypt" }
```

Get All

```
GET /users
```

Get One

```
GET /users/USER_ID
```

Delete

```
DELETE /users/USER_ID
```

9. Quick Request Examples

Create

```
POST /users
```

```
{ "username": "Ali", "email": "ali@mail.com", "country":
```

```
"Egypt" }
```

Get All

```
GET /users
```

Get One

```
GET /users/USER_ID
```

Delete

```
DELETE /users/USER_ID
```

10. الخلاصة

لأنك ترى بوضوح كيف يبدأ التطبيق، وكيف يربط الـ controller بين الـ module، وكيف يتم تسجيل الـ NestJS وهذا المشروع ممتاز كبداية لفهم

والمنطق. الجزء الأهم هنا هو أن كل شيء بسيط ومباشر، لذلك هو

routes

Service, Validation,

ب للتعلم قبل الانتقال إلى

Database.

10. Summary

This project is a good starting point for learning NestJS because it clearly

shows how the app starts, how modules are registered, and how a

controller connects routes to logic. Its simplicity makes it useful for learning

before moving to `Service`, `Validation`, and `Database` layers.